WNDRVR
SUPPORT NETWORK

VXWORKS 7 LAYER VERSION MANAGEMENT TUTORIAL, SR0650

Copyright Notice

Copyright © 2022 Wind River Systems, Inc.

All rights reserved. No part of this publication may be reproduced or transmitted in any form or by any means without the prior written permission of Wind River Systems, Inc.

Wind River, Simics, Tornado, and VxWorks are registered trademarks of Wind River Systems, Inc. Helix, Pulsar, Rocket, Titanium Cloud, Titanium Control, Titanium Core, Titanium Edge, Titanium Edge SX, Titanium Server, and the Wind River logo are trademarks of Wind River Systems, Inc. Any third-party trademarks referenced are the property of their respective owners. For further information regarding Wind River trademarks, please see:

www.windriver.com/company/terms/trademark.html

This product may include software licensed to Wind River by third parties. Relevant notices (if any) are provided for your product on the Wind River download and installation portals:

https://delivers.windriver.com/

https://windshare.usa.windriver.com/

Wind River may refer to third-party documentation by listing publications or providing links to third-party websites for informational purposes. Wind River accepts no responsibility for the information provided in such third-party documentation.

Corporate Headquarters

Wind River
500 Wind River Way
Alameda, CA 94501-1153
U.S.A.
Toll free (U.S.A.): +1-800-545-WIND
Telephone: +1-510-748-4100
Facsimile: +1-510-749-2010

For additional contact information, see the Wind River website:

www.windriver.com

For information on how to contact Customer Support, see:

www.windriver.com/support

VxWorks 7 Layer Version Management Tutorial, SR0650

10 June 2020

1. LAYER VERSION MANAGEMENT TUTORIAL

You can modify both custom and VxWorks® supplied layers and use the modified layers in a VxWorks source build (VSB).

For this tutorial the workflow is as follows:

1. Create a custom layer.
2. Create a new version of a custom layer.
3. Test the new layer on a target.
4. Create a new verion of a Wind River supplied layer.
5. Test the customized Wind River layer on a target.

2. EXAMPLE: CREATING AN INSIDE CUSTOM LAYER AND INTEGRATING IT INTO A VSB

You can create a custom layer within the VxWorks source tree and integrate it into your VxWorks source build (VSB) project using the project build system.

Before You Begin

This example assumes you have access to the **cobble** downloadable kernel module (DKM) example project at *installDir* **\vxworks-7\samples\dkm**.

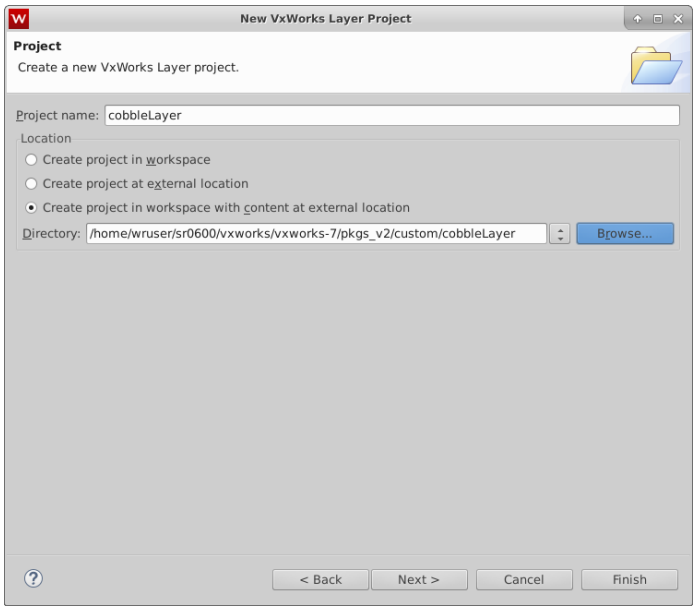
It also assumes you have a pre-existing VxWorks source build (VSB) and VxWorks image project (VIP) present in Workbench. Follow the steps below using a VxWorks simulator (VxSim) instance as the target.

About This Task

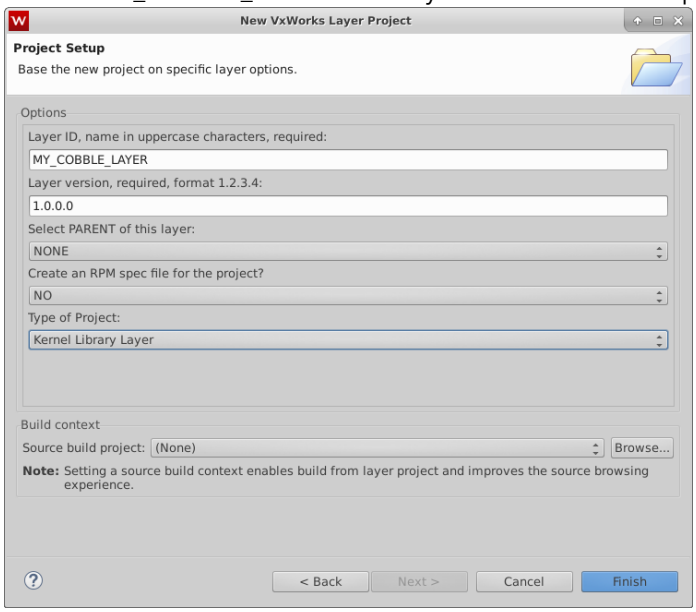
For this tutorial, you use the Workbench **cobble** example DKM project to represent the application code you want to integrate into a layer. Create a layer project, insert the **cobble** DKM application code into it, rebuild the VSB and VIP, and check that the target boots and runs.

Procedure

1. Create a folder for your new layer projects.
    1. Create a root directory within the Wind River installation at *installDir* **\vxworks-7\pkgs_v2\custom**.
    2. Create a folder for your new custom cobble layer project at *installDir* **\vxworks-7\pkgs_v2\custom\cobbleLayer**.
2. Create a VxWorks layer project in your new layer project folder and call it **cobbleLayer**.
    1. From the Workbench menu, select **File > New > Wind River Workbench Project**.
    2. Select the option VxWorks 7 as the target operating system from the drop-down menu if you are prompted to choose the **Target Operating System**.
    3. From the Build type drop-down list, select **Layer**. Click **Next**.
    4. Give the Project the name **cobbleLayer**.
    5. Under the **Location** dialog, select **Create project in workspace with content at external location** and browse to the layer folder. Click **Next**.



6. Enter **MY_COBBLE_LAYER** as the layer ID. Leave all other options as default and click **Finish**.



3. Populate **cobbleLayer** with **cobble.c**.
    1. Go to *installDir* **\vxworks-7\samples\dkm\cobble**.
    2. Right-click on the **cobble.c** source file and select **Copy**.
    3. In the project explorer, navigate to **cobbleLayer**. Highlight the **src** folder within the **cobbleLayer** project and click **Paste**.
4. Update the **cobbleLayer** metadata file.
    1. Locate the file **layer.vsbl** at the root of the **cobbleLayer** project. Open the file for editing.
    2. Locate the line beginning with the tag **SYNOPSIS**. Replace the remaining text on this line with an appropriate description. For example, "This layer implements Cobble functionality".

```
Layer MY_COBBLE_LAYER {
SYNOPSIS This layer implements Cobble functionality
VERSION 1.0.0.0
```

    3. Save the file.

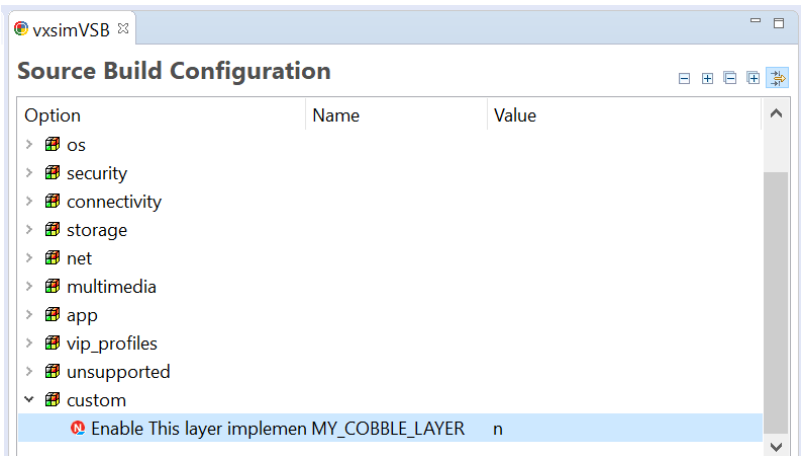
5. Use a host terminal to check that **cobbleLayer**, otherwise known as **MY_COBBLE_LAYER**, does exist.

```
C:\WindRiver\> wrenv -p vxworks-7
C:\WindRiver\> wrtool -data C:\WindRiver\workspace
Active workspace is 'C:\WindRiver\workspace'.
WrTool, version 4.1.0
Copyright (c) 2013 - 2014 Wind River Systems, Inc.


C:\WindRiver\> prj vsb info MY_COBBLE_LAYER
Name: MY_COBBLE_LAYER
[...]
[...]
Location: C:/WindRiver/vxworks-7/pkgs_v2/custom/cobbleLayer
```

6. Verify that your VSB has access to **MY_COBBLE_LAYER** by opening the VSB source build configuration in the project explorer.

   You can see **MY_COBBLE_LAYER** appearing as a component within the Source Build Configuration and the **custom** container.



7. Include **cobbleLayer** in your VSB.
   1. Right-click on the red component within **custom** and select **Add with Dependencies** to add **MY_COBBLE_LAYER** to the VSB.
   2. Observe **MY_COBBLE_LAYER** is now a green component.
   3. In the project explorer, right-click on the VSB and select **Rebuild Project**.
8. Modify your VIP to start the cobble application upon target startup.
   1. Locate the VIP related to your VSB in the project explorer.
   2. Locate the **usrAppInit.c** file within the project and open it for editing.
   3. Insert a call to the **cobble.c** progStart( ) function into the **usrAppInit.c** usrAppInit( ) function.

```
/*******************************************************************************
*
* usrAppInit - initialize the users application
*/

void usrAppInit (void)
    {
#ifdef  USER_APPL_INIT
    USER_APPL_INIT;     /* for backwards compatibility */
#endif

    /* TODO: add application specific code here */
    progStart();
    }
```

9. Rebuild the VIP and reboot the target from the rebuilt VIP. Check that the expected cobble tasks are running on the target from the VxWorks shell.

   The **cobble.c** file is a simple multi-tasking application that simulates a data collection system in which data is retrieved from an external source (for example, a device that generates interrupts as data comes in). The four cobble tasks are:

   - tCosmos, which simulates an interrupt service routine (ISR) and generates new data.
   - tSchlep, which collects the data.
   - tCrunch, which performs calculations on the data to obtain a result value.
   - tMonitor, which monitors the results and prints a warning message if they are outside of the safety range.

   > **Note:** If you do not add progStart( ) to **usrAppInit.c**, cobble does not start upon target reboot and **cobbleLayer** does not link into the kernel image. Rather, the layer is omitted by the VxWorks build system because there is no reference to the **cobble.c** application in the VxWorks BSP or application code. As a consequence, the VxWorks build system linker omits **cobbleLayer** during the link phase of the VIP build.

```
-> i

  NAME         ENTRY       TID    PRI   STATUS      PC       SP     ERRNO  DELAY
----------  ------------ -------- --- ---------- -------- -------- ------- -----
tIsr0       102678f0     3800da08   0 PEND       102714ee 380cff60       0     0
tJobTask    101a82b0     38049e08   0 PEND       102714ee 3813ff00       0     0
tExcTask    101a7ca0     103dfb40   0 PEND       102714ee 103e5d40       0     0
tLogTask    logTask      3804dd48   0 PEND       1026f073 3817feb0       0     0
tShell0     shellTask    38369d40   1 READY      1027a760 3841fc40       0     0
tErfTask    100ed8c0     38002e10  10 PEND       10271e1f 3802ff00       0     0
ipcom_tick> 1003a1b0     38437e48  20 PEND       102714ee 384dfed0       0     0
tVxdbgTask  100ffc70     3832b4b0  25 PEND       102714ee 3838ff20       0     0
tTcfEvents  1020f1f0     103f8380  49 PEND+T     102714ee 103f81f0       0    54
tTcf        1015a0b0     383ffb18  49 PEND       102714ee 3845f890       0     0
tTcf        1015a0b0     38433a10  49 PEND       102714ee 3849f8a0       0     0
tTcf        1015a0b0     38479908  49 PEND       102714ee 3861f8f0       0     0
tTcf        1015a0b0     3847d5c8  49 PEND       102714ee 3865fa40       0     0
tNet0       ipcomNetTask 38008b30  50 PEND       102714ee 3806fef0       0     0
tAioIoTask> aioIoTask    3811b440  50 PEND       10271e1f 381ffef0       0     0
tAioIoTask> aioIoTask    3811e870  50 PEND       10271e1f 3823fef0       0     0
ipcom_sysl> 10038c10     38086280  50 PEND       10271e1f 382bfe20       0     0
tNetConf    10075840     380a5ec8  50 PEND       102714ee 3834fcf0       0     0
tAnalysisA> cafe_event_> 103d9ca0  50 PEND       102714ee 103d9b30       0     0
tAioWait    aioWaitTask  38117f10  51 PEND       102714ee 381bfe80       0     0
tCosmos     cosmos       38438518 200 DELAY      10279812 3851ff80       0     1
tSchlep     schlep       38438948 220 PEND       102714ee 3855ff20       0     0
tMonitor    monitor      3843f900 230 READY      10149800 385dffb0       0     0
tCrunch     crunch       3843f5d0 240 READY      100b5ae0 3859ffd0       0     0
ipcom_egd   10037c10     38089998 255 READY      10037c80 382ffec0       0     0
value = 0 = 0x0
->
```

3. EXAMPLE: CREATING AN OUTSIDE CUSTOM LAYER AND INTEGRATING IT INTO A VSB

You can create a custom layer externally to the VxWorks source tree and integrate it into your VxWorks source build (VSB) project using the project build system.

Before You Begin

This example assumes the Workbench project explorer already contains a **cobble** downloadable kernel module (DKM) example project.

It also assumes you have a pre-existing VxWorks source build (VSB) and VxWorks image project (VIP) present in Workbench. Follow the steps below using a VxWorks simulator (VxSim) instance as the target.

This tutorial also assumes you have no previously created **cobbleLayer** projects, such as the one developed in **Example: Creating an Inside Custom Layer and Integrating it into a VSB**. Before you begin, remove any existing **cobbleLayer** projects, if necessary, by deleting them from the filesystem.

About This Task

For this tutorial, you use the Workbench **cobble** example DKM project to represent the application code that you want to integrate into the layer. Create a layer project, insert the **cobble** DKM application code into it, rebuild the VSB and VIP, and check that the target boots and runs.

Procedure

1. Create a folder for your new layer project.

   In this example, you are creating an outside custom layer. Therefore, the path to this folder must be outside the Wind River installation. The layer project folder should not be positioned within *installDir* **/VxWorks-7/pkgs_v2**. Instead, it belongs externally; for example, in **C:/MyLayers/cobbleLayer**.

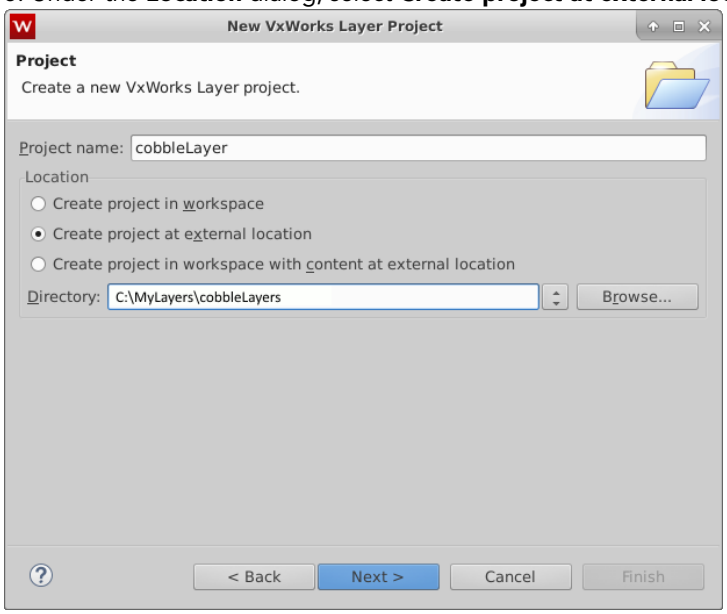
2. Edit the *WIND_LAYER_PATHS* environment variable and add the full path to your new layer project folder. This layer path is **C:\MyLayers\cobbleLayer**.
   1. Go to **Control Panel > System**.
   2. In the left pane, click on **Advanced system settings**.
   3. On the Advanced pane, click **Environment Variables**.
   4. In the **System Variables** area, add a new variable called *WIND_LAYER_PATHS*.
   5. Set this new variable to your custom path. Click OK. Save and exit as appropriate.

   Note: If *WIND_LAYER_PATHS* exists, and already contains a path, append a semicolon to the pre-existing path and insert the new custom path after the semicolon for Windows systems. For Linux systems, use colon as path separators. Do not leave space before the new custom path.

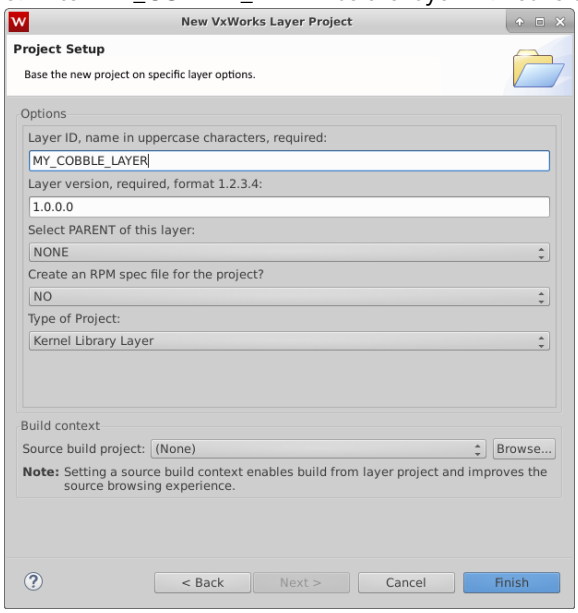
   Note: You may need to reboot your host workstation in order for the *WIND_LAYER_PATHS* environment variable change to take place.

3. Create a VxWorks layer project in your new layer project folder and call it **cobbleLayer**.
   1. From the Workbench menu, select **File > New > Wind River Workbench Project**.
   2. Select the option VxWorks 7 as the target operating system from the drop-down menu if you are prompted to choose the **Target Operating System**.

3. From the Build type drop down list, select **Layer**. Click **Next**.
4. Name the Project **cobbleLayer**.
5. Under the **Location** dialog, select **Create project at external location** and browse to the layer folder. Click **Next**.



6. Enter **MY_COBBLE_LAYER** as the layer ID. Leave all other options as default and click **Finish**.



4. Populate **cobbleLayer** with **cobble.c**.
    1. In the Workbench project explorer, expand the **cobble** project.
    2. Right-click on **cobble.c** and select **Copy**.
    3. In the project explorer, navigate to **cobbleLayer**. Highlight the **src** folder within the **cobbleLayer** project and click **Paste**.
5. Use a host terminal to check that **cobbleLayer** exists.

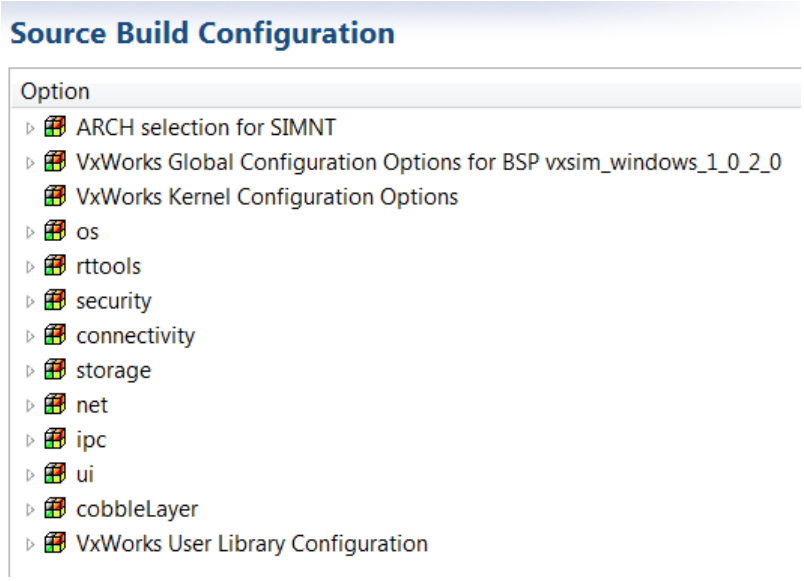
```
C:\WindRiver\workspace> wrenv -p vxworks-7
C:\WindRiver\> wrtool -data C:\WindRiver\workspace
Active workspace is 'C:\WindRiver\workspace'.
WrTool, version 4.1.0
```

```
Copyright (c) 2013 - 2014 Wind River Systems, Inc.


workspace> prj vsb info MY_COBBLE_LAYER
Name:                       MY_COBBLE_LAYER
VERSION:                    1.0.0.0
[...]
Location: C:/MyLayers/cobbleLayer
```

6. Verify that your VSB has access to **cobbleLayer** by opening the VSB source build configuration in the project explorer.

   You can see **cobbleLayer** appearing there as a root component in the Source Build Configuration.



7. Include **cobbleLayer** in your VSB.
   1. Right-click on the red component within **cobbleLayer** and select **Add with Dependencies** to add it to the VSB.
   2. In the project explorer, right-click on the VSB and select **Rebuild Project**.
8. Modify your VIP to start the cobble application upon target startup.
   1. Locate the VIP related to your VSB in the project explorer.
   2. Locate the **usrAppInit.c** file within the project and open it for editing.
   3. Insert a call to the **cobble.c** progStart( ) function into the **usrAppInit.c** usrAppInit( ) function.

```
/*******************************************************************************
*
* usrAppInit - initialize the users application
*/

void usrAppInit (void)
    {
#ifdef  USER_APPL_INIT
    USER_APPL_INIT;     /* for backwards compatibility */
#endif

    /* TODO: add application specific code here */
    progStart();
    }
```

9. Rebuild the VIP and reboot the target from the rebuilt VIP. Check that the expected cobble tasks are running on the target from the VxWorks shell.

   The **cobble.c** file is a simple multi-tasking application that simulates a data collection system in which data is retrieved from an external source (for example, a device that generates interrupts as data comes in). The four cobble tasks are:

- tCosmos, which simulates an interrupt service routine (ISR) and generates new data.
- tSchlep, which collects the data.
- tCrunch, which performs calculations on the data to obtain a result value.
- tMonitor, which monitors the results and prints a warning message if they are outside of the safety range.

> **Note:** If you do not add progStart( ) to **usrAppInit.c**, cobble does not start upon target reboot and **cobbleLayer** does not link into the kernel image. Rather, the layer is omitted by the VxWorks build system because there is no reference to the **cobble.c** application in the VxWorks BSP or application code. As a consequence, the VxWorks build system linker omits **cobbleLayer** during the link phase of the VIP build.

```
-> i

  NAME         ENTRY       TID    PRI   STATUS      PC       SP     ERRNO  DELAY
----------  ------------ -------- --- ---------- -------- -------- ------- -----
tIsr0       102678f0     3800da08   0 PEND       102714ee 380cff60       0     0
tJobTask    101a82b0     38049e08   0 PEND       102714ee 3813ff00       0     0
tExcTask    101a7ca0     103dfb40   0 PEND       102714ee 103e5d40       0     0
tLogTask    logTask      3804dd48   0 PEND       1026f073 3817feb0       0     0
tShell0     shellTask    38369d40   1 READY      1027a760 3841fc40       0     0
tErfTask    100ed8c0     38002e10  10 PEND       10271e1f 3802ff00       0     0
ipcom_tick> 1003a1b0     38437e48  20 PEND       102714ee 384dfed0       0     0
tVxdbgTask  100ffc70     3832b4b0  25 PEND       102714ee 3838ff20       0     0
tTcfEvents  1020f1f0     103f8380  49 PEND+T     102714ee 103f81f0       0    54
tTcf        1015a0b0     383ffb18  49 PEND       102714ee 3845f890       0     0
tTcf        1015a0b0     38433a10  49 PEND       102714ee 3849f8a0       0     0
tTcf        1015a0b0     38479908  49 PEND       102714ee 3861f8f0       0     0
tTcf        1015a0b0     3847d5c8  49 PEND       102714ee 3865fa40       0     0
tNet0       ipcomNetTask 38008b30  50 PEND       102714ee 3806fef0       0     0
tAioIoTask> aioIoTask    3811b440  50 PEND       10271e1f 381ffef0       0     0
tAioIoTask> aioIoTask    3811e870  50 PEND       10271e1f 3823fef0       0     0
ipcom_sysl> 10038c10     38086280  50 PEND       10271e1f 382bfe20       0     0
tNetConf    10075840     380a5ec8  50 PEND       102714ee 3834fcf0       0     0
tAnalysisA> cafe_event_> 103d9ca0  50 PEND       102714ee 103d9b30       0     0
tAioWait    aioWaitTask  38117f10  51 PEND       102714ee 381bfe80       0     0
tCosmos     cosmos       38438518 200 DELAY      10279812 3851ff80       0     1
tSchlep     schlep       38438948 220 PEND       102714ee 3855ff20       0     0
tMonitor    monitor      3843f900 230 READY      10149800 385dffb0       0     0
tCrunch     crunch       3843f5d0 240 READY      100b5ae0 3859ffd0       0     0
ipcom_egd   10037c10     38089998 255 READY      10037c80 382ffec0       0     0
value = 0 = 0x0
->
```

4. EXAMPLE: CREATING A NEW VERSION OF AN ESTABLISHED CUSTOM LAYER

Multiple versions of a VxWorks layer can be developed in parallel and can safely coexist in Workbench.

Before You Begin

This example assumes you have already completed Example: Creating an Inside Custom Layer and Integrating it into a VSB on page 2. This example builds on that tutorial.

This example also assumes that you have a pre-existing VxWorks source build (VSB) and VxWorks image project (VIP) present in Workbench. You use a VxWorks simulator (VxSim) instance as the target.

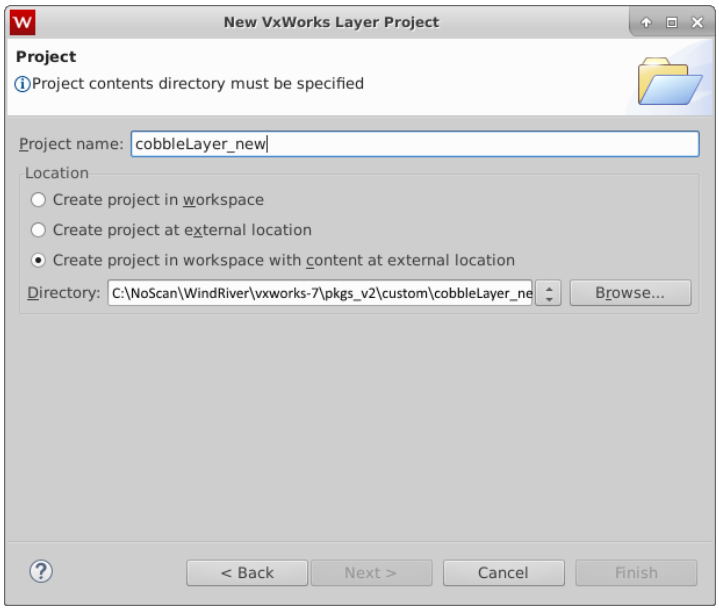
About This Task

Wind River recommends that, when developing multiple iterations of a custom layer, you keep the layer in its original setting. That is, do not mix inside and outside locations for the custom layer. This example continues the development of an inside custom layer.

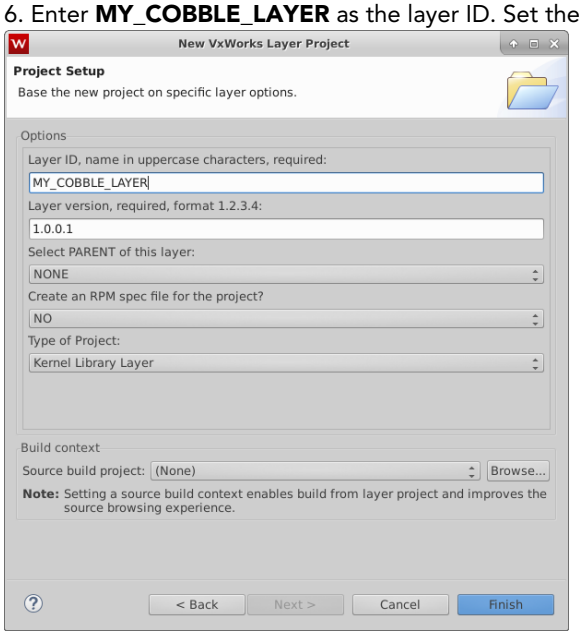
In this example, you use the VxWorks layer project **cobbleLayer** and you fix a defect in the source code, moving the layer project version from 1.0.0.0 to 1.0.0.1. You then migrate the VSB to use **cobbleLayer** version 1.0.0.1, rebuild the VIP, and check that the target boots and runs.

Procedure

1. Create a folder for your new layer project at *installDir* **\vxworks-7\pkgs_v2\custom\cobbleLayer_new**.
2. Create a VxWorks layer project.
    1. From the Workbench menu, select **File > New > Wind River Workbench Project**.
    2. Select the option VxWorks 7 as the target operating system from the drop-down menu if you are prompted to choose the **Target Operating System**.
    3. From the Build type drop down list, select **Layer**. Click **Next**.
    4. Give the project the name **cobbleLayer_new**.
    5. Under the **Location** dialog, select **Create project in workspace with content at external location** and browse to the layer folder. Click **Next**.



6. Enter **MY_COBBLE_LAYER** as the layer ID. Set the layer version to 1.0.0.1. Leave all other options as default. Click **Finish**.

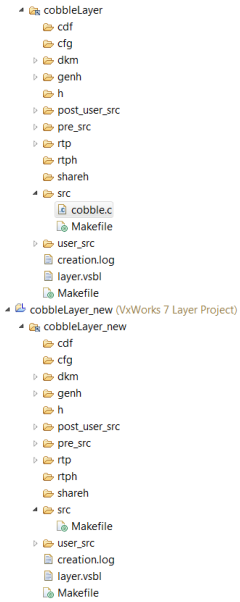


3. Update the **cobbleLayer_new** metadata file.
   1. Locate the **layer.vsbl** file at the root of the **cobbleLayer** project. Open the file for editing.
   2. Locate the line beginning with the tag **SYNOPSIS**. Replace the remaining text on this line with an appropriate description. For example, "This layer implements Cobble functionality".

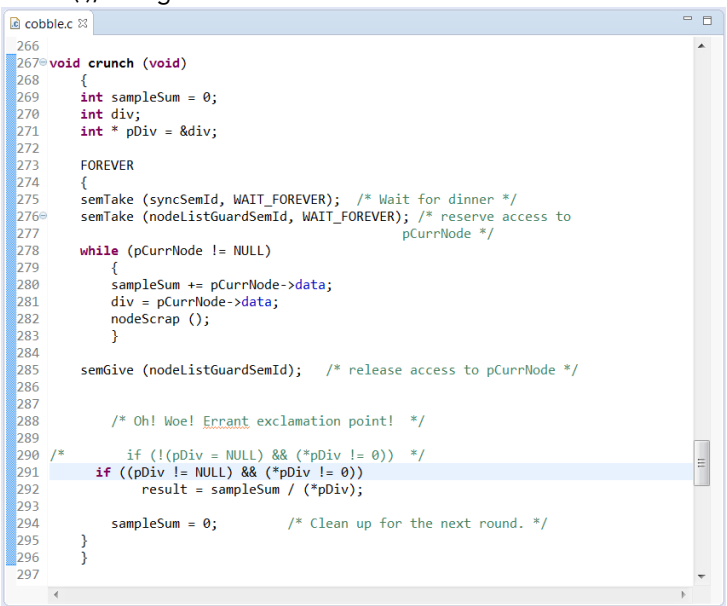
```
Layer MY_COBBLE_LAYER {
SYNOPSIS This layer implements Cobble functionality
VERSION 1.0.0.1
```

   3. Save the file.
4. Populate **cobbleLayer_new** with **cobble.c** from **cobbleLayer**.
   1. In the project explorer view, expand the **cobbleLayer** project you created in the previous tutorial example. Identify the folder named **src** containing **cobble.c**.
   2. Right-click on the **cobble.c** source file and select **Copy**.

3. Navigate to **cobbleLayer_new** in the project explorer. Identify the folder named **src**, which is currently empty apart from a **Makefile**. Highlight **src** and select **Paste**.



5. Correct a defect in the **cobbleLayer_new** project in the **cobble.c** file.
   1. Locate **cobble.c** and open the file for editing.
   2. Locate the function crunch( ) at line 267 of **cobble.c**. In order to resolve a possible divide by zero exception condition in crunch( ), change the code as follows on line 290 and 291.



```
cobble.c
266
267 void crunch (void)
268     {
269     int sampleSum = 0;
270     int div;
271     int * pDiv = &div;
272
273     FOREVER
274     {
275     semTake (syncSemId, WAIT_FOREVER);  /* Wait for dinner */
276     semTake (nodeListGuardSemId, WAIT_FOREVER); /* reserve access to
277                                                    pCurrNode */
278     while (pCurrNode != NULL)
279         {
280         sampleSum += pCurrNode->data;
281         div = pCurrNode->data;
282         nodeScrap ();
283         }
284
285     semGive (nodeListGuardSemId);   /* release access to pCurrNode */
286
287
288         /* Oh! Woe! Errant exclamation point!  */
289
290 /*        if (!(pDiv = NULL) && (*pDiv != 0))  */
291      if ((pDiv != NULL) && (*pDiv != 0))
292            result = sampleSum / (*pDiv);
293
294         sampleSum = 0;         /* Clean up for the next round. */
295     }
296     }
297
```

   3. From the Workbench menu, select **File > Save**.
6. Use a host terminal to verify that the layers **cobbleLayer** and **cobbleLayer_new** both exist.

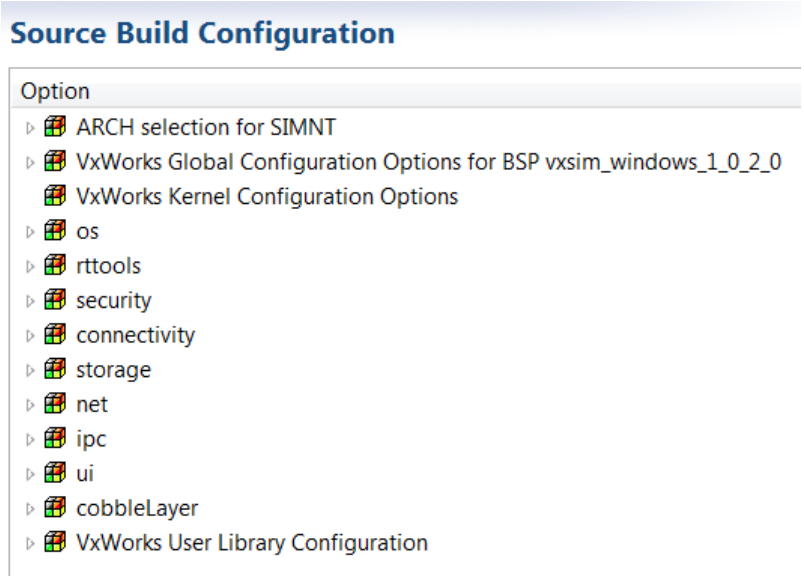
```
C:\WindRiver\> wrenv -p vxworks-7
C:\WindRiver\> wrtool -data C:\WindRiver\workspace
C:\WindRiver\> prj vsb info MY_COBBLE_LAYER
Name: MY_COBBLE_LAYER
[...]
[...]
```

```
Location: C:/WindRiver/vxworks-7/pkgs_v2/custom/cobbleLayer
[...]
Name: MY_COBBLE_LAYER
[...]
Location: C:/WindRiver/vxworks-7/pkgs_v2/custom/cobbleLayer_new
```

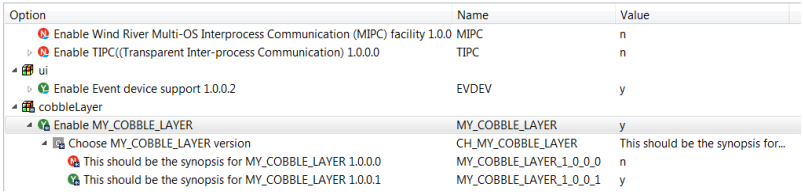
Note: Notice the identifying features for both version 1.0.0.0 and version 1.0.0.1 of **cobbleLayer** in the resulting output.

7. Verify that **cobbleLayer** is available to your VSB by opening the VSB source build configuration in the project explorer.

   You can see **cobbleLayer** appearing there as a root component in the Source Build Configuration.



8. Enable version 1.0.0.1 of **cobbleLayer** in your VSB.
   1. Verify that the component entitled **Enable MY_COBBLE_LAYER** is green, indicating it is already included in the VSB. If it is red, right-click on the red **Enable MY_COBBLE_LAYER** and select **Add with Dependencies** to add it to the VSB.
   2. Beneath **Enable MY_COBBLE_LAYER**, select **Choose MY_COBBLE_LAYER version**. Right-click on the red component and select **Change Value** to ensure that only **MY_COBBLE_LAYER 1.0.0.1** is set to green and therefore included in the VSB.
   3. Right-click on the VSB in the project explorer and select **Rebuild Project**.



Option	Name	Value
Enable Wind River Multi-OS Interprocess Communication (MIPC) facility 1.0.0	MIPC	n
Enable TIPC((Transparent Inter-process Communication) 1.0.0.0	TIPC	n
ui		
Enable Event device support 1.0.0.2	EVDEV	y
cobbleLayer		
Enable MY_COBBLE_LAYER	MY_COBBLE_LAYER	y
Choose MY_COBBLE_LAYER version	CH_MY_COBBLE_LAYER	This should be the synopsis for...
This should be the synopsis for MY_COBBLE_LAYER 1.0.0.0	MY_COBBLE_LAYER_1_0_0_0	n
This should be the synopsis for MY_COBBLE_LAYER 1.0.0.1	MY_COBBLE_LAYER_1_0_0_1	y

9. Modify your VIP to start the cobble application upon target startup.
   1. Identify the VIP that related to your VSB in the project explorer.
   2. Locate the **usrAppInit.c** file within the project and open for editing.
   3. Insert a call to the **cobble.c** function progStart( ) into the **usrAppInit.c** function usrAppInit( ).

```
/*******************************************************************************
*
* usrAppInit - initialize the users application
*/

void usrAppInit (void)
    {
#ifdef  USER_APPL_INIT
    USER_APPL_INIT;     /* for backwards compatibility */
#endif

    /* TODO: add application specific code here */
    progStart();
    }
```

10. Rebuild the VIP and reboot the target from the rebuilt VIP. Check that the expected cobble tasks are running on the target from the VxWorks shell.

    The **cobble.c** application is a simple multi-tasking application that simulates a data collection system in which data is retrieved from an external source (for example, a device that generates interrupts as data comes in). The four cobble tasks are:

    - tCosmos, which simulates an interrupt service routine (ISR) and generates new data.
    - tSchlep, which collects the data.
    - tCrunch, which performs calculations on the data to obtain a result value.
    - tMonitor, which monitors the results and prints a warning message if they are outside of the safety range.

> **Note:** If you do not add progStart( ) to **usrAppInit.c**, the **cobbleLayer** does not start upon target reboot because it does not link into the kernel image. Rather, the layer is omitted by the VxWorks build system because there is no reference to the **cobble.c** application in the VxWorks BSP or application code. As a consequence, the VxWorks build system linker omits **cobbleLayer** during the link phase of the VIP build.

```
-> i

  NAME         ENTRY       TID    PRI   STATUS      PC       SP     ERRNO  DELAY
----------  ------------ -------- --- ---------- -------- -------- ------- -----
tIsr0       102678f0     3800da08   0 PEND       102714ee 380cff60       0     0
tJobTask    101a82b0     38049e08   0 PEND       102714ee 3813ff00       0     0
tExcTask    101a7ca0     103dfb40   0 PEND       102714ee 103e5d40       0     0
tLogTask    logTask      3804dd48   0 PEND       1026f073 3817feb0       0     0
tShell0     shellTask    38369d40   1 READY      1027a760 3841fc40       0     0
tErfTask    100ed8c0     38002e10  10 PEND       10271e1f 3802ff00       0     0
ipcom_tick> 1003a1b0     38437e48  20 PEND       102714ee 384dfed0       0     0
tVxdbgTask  100ffc70     3832b4b0  25 PEND       102714ee 3838ff20       0     0
tTcfEvents  1020f1f0     103f8380  49 PEND+T     102714ee 103f81f0       0    54
tTcf        1015a0b0     383ffb18  49 PEND       102714ee 3845f890       0     0
tTcf        1015a0b0     38433a10  49 PEND       102714ee 3849f8a0       0     0
tTcf        1015a0b0     38479908  49 PEND       102714ee 3861f8f0       0     0
tTcf        1015a0b0     3847d5c8  49 PEND       102714ee 3865fa40       0     0
tNet0       ipcomNetTask 38008b30  50 PEND       102714ee 3806fef0       0     0
tAioIoTask> aioIoTask    3811b440  50 PEND       10271e1f 381ffef0       0     0
tAioIoTask> aioIoTask    3811e870  50 PEND       10271e1f 3823fef0       0     0
ipcom_sysl> 10038c10     38086280  50 PEND       10271e1f 382bfe20       0     0
tNetConf    10075840     380a5ec8  50 PEND       102714ee 3834fcf0       0     0
tAnalysisA> cafe_event_> 103d9ca0  50 PEND       102714ee 103d9b30       0     0
tAioWait    aioWaitTask  38117f10  51 PEND       102714ee 381bfe80       0     0
tCosmos     cosmos       38438518 200 DELAY      10279812 3851ff80       0     1
tSchlep     schlep       38438948 220 PEND       102714ee 3855ff20       0     0
tMonitor    monitor      3843f900 230 READY      10149800 385dffb0       0     0
tCrunch     crunch       3843f5d0 240 READY      100b5ae0 3859ffd0       0     0
ipcom_egd   10037c10     38089998 255 READY      10037c80 382ffec0       0     0
value = 0 = 0x0
->
```

5. EXAMPLE: CUSTOMIZING A WIND RIVER LAYER

VxWorks is composed of Wind River-supplied layers. You can modify and customize standard layers in a controllable and manageable way. Do not modify the Wind River installation directories, source files, and header files when customizing standard layers.

Before You Begin

This example assumes that you have already completed Example: Creating an Outside Custom Layer and Integrating it into a VSB on page 6.

About This Task

This example uses a Wind River-supplied tools layer, **rttools-2.0.0.0**, which contains the VxWorks debug agent, **agent** which is currently included in the VxWorks source build configuration. This example gives the workflow for modifying the function of **agent** without modifying any of the original layer's source code.

> Note: At the time of this writing, the tools layer is known as **rttools-2.0.0.0**. The version number may increment by the time you start this example. Translate the instructions as needed.

Procedure

1. Identify the **Enable VxWorks RTTOOLS layer** component in your VSB.
   1. In the Workbench project explorer, open the VSB source build configuration.
   2. Open the **debug_analysis** root component and notice that the **Enable VxWorks RTTOOLS layer** is included.
2. Take a copy of **rttools-2.0.0.0** and prepare to modify it.
   1. Open a Windows Explorer and navigate to *installDir* **\vxworks-7\pkgs_v2\os\debug_analysis\rttools-2.0.0.0**.
   2. From the Windows Explorer, copy the folder **rttools-2.0.0.0** and rename the resulting folder as **rttools-2.0.0.1**.
   3. Open the file **rttools-2.0.0.1\layer.vsbl** in the Workbench editor. Identify the *VERSION* line. Increment that version number to 2.0.0.1.

```
Layer RTTOOLS {
SYNOPSIS     VxWorks RTTOOLS layer
HELP         This provides a number of libraries DEBUG_AGENT, RUNTIME_ANALYSIS, STACKTRACE,
STOP_MODE_DEBUG_AGENT, and SYSTEMVIEWER_AGENT.
VERSION      2.0.0.1
```

3. Reposition **rttools-2.0.0.1** as an outside layer. From the Windows explorer, move the newly created folder **rttools-2.0.0.1** to **C:\MyLayers\rttools-2.0.0.1**.

   > Note: In order to decouple our modifications from Wind River's delivered layers, treat your new modified agent as an outside layer.

4. Edit the *WIND_LAYER_PATHS* environment variable and add the full path to your new layer project folder. This layer path is **C:\MyLayers\rttools-2.0.0.1**.
   1. Go to **Control Panel > System**.
   2. In the left pane, click on **Advanced system settings**.
   3. On the Advanced pane, click **Environment Variables**.
   4. In the **System Variables** area, locate the variable called *WIND_LAYER_PATHS*.
   5. Append your custom path to *WIND_LAYER_PATHS*. Click OK Save and exit as appropriate.

> **Note:** If *WIND_LAYER_PATHS* already contains a path, append a semicolon to the pre-existing path and insert the new custom path after the semicolon. Do not put a space before the new custom path.

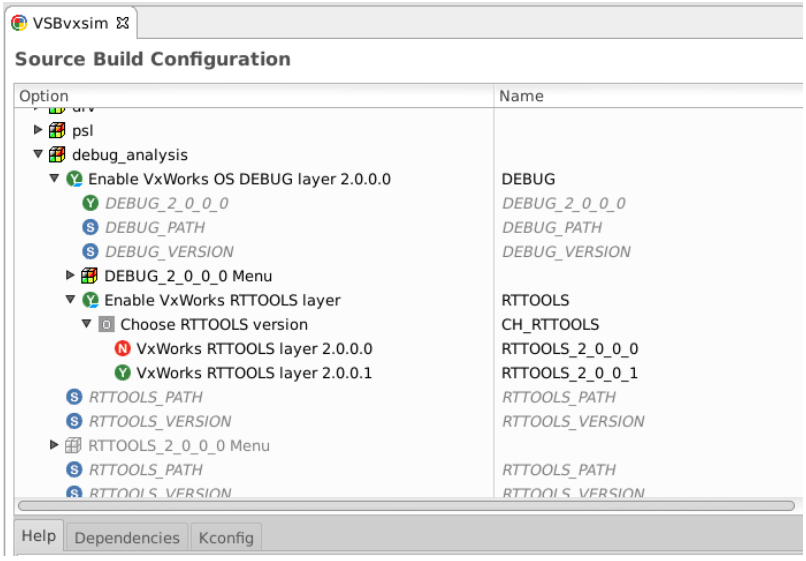
> **Note:** You may need to reboot your host workstation in order for the *WIND_LAYER_PATHS* environment variable change to take place.

5. Modify the function of the debug agent **rttools-2.0.0.1\agent**.
   1. Open **C:\MyLayers\rttools-2.0.0.1\agent\src\osconfig\vxworks\cfg\usrDebugAgent.c** in the Workbench editor.
   2. Locate the usrDebugAgentBannerInit( ) function. Find the string "Started (always)" and modify this string to say "Started (always) and modified by Me".

```
/*******************************************************************************
*
* usrDebugAgentBannerInit - register the debug agent banner with usrBanner
*
* Add agent specific data to the VxWorks banner.
*
* RETURNS : N/A
*
* NOMANUAL
*/

void usrDebugAgentBannerInit (void)
    {
#ifdef INCLUDE_DEBUG_AGENT_START
    (void) bannerItemAdd     ("        Debug Agent", "Started (always) and modified by Me");
#else
    if (sysDebugModeGet ())
    {
    (void) bannerItemAdd ("        Debug Agent", "Started (boot flag)");
    }
    else
    {
    (void) bannerItemAdd ("        Debug Agent", "Not started");
    }
#endif
    }
#endif
```

6. Update your VSB to use the new tools layer, **rttools-2.0.0.1**, containing the updated debug agent .
   1. In the project explorer, open the VSB source build configuration.
   2. Open the **debug_analysis** root component and notice that **Enable VxWorks RTTOOLS layer** is still included.
   3. The Source Build Configuration gives a choice to change the version of this layer. Choose your newly modified layer, **RTTOOLS_2_0_0_1**.
   4. Right-click on **RTTOOLS_2_0_0_1** and select **Change Value**. This activates the new layer within the VSB.



7. Rebuild the VSB. Once this has completed, rebuild the VIP and reboot the target.

8. Notice on the target console that the agent status output string has changed. Your VIP is now running the updated debug agent in **rttools-2.0.0.1**.

```
 ________          ________
 \77777777\        /77777777/
  \77777777\      /77777777/
   \77777777\    /77777777/
    \77777777\  /77777777/
     \77777777\ \7777777/
      \77777777\  \77777/            VxWorks 7
       \77777777\  \777/
        \77777777\  \7/     Core Kernel version: 1.0.10.2
         \77777777\  -      Build date: Jun 17 2015 15:21:04
          \77777777\
           \7777777/        Copyright Wind River Systems, Inc.
            \77777/  -                1984-2015
             \777/  /7\
              \7/  /777\
               -  -------

             Board: SIMNT board
     OS Memory Size: 511MB
   ED&R Policy Mode: Deployed
       Debug Agent: Started (always) and modified by Me

Loading symbol table from host:C:/NoScan/WindRiver/workspace/vip_sim/default/vxWorks.sym ...done

-> 
```

9. To revert your VSB to the original Wind River debug layer, do the following:
   1. In the VSB Source Build Configuration , right-click on **rttools-2.0.0.0** and select **Change Value**.
   2. Rebuild the VSB. This reverts your VSB back to the original Wind River-supplied debug agent layer.
   3. Rebuild the VIP and reboot the target to check that the debug agent functionality has reverted back to the original behaviour.